

Automatización Git y Sistema de Ventas

Taller de Tecnologías 1

Docente: Natalia Jauregui

Fabrizio Pedemonti N°372959

Lucas Pittaluga N°372926

Agustin Roizen N°350021

Indice

1. Introduccion
2. Organización del Trabajo
3. Inicialización del repositorio y GitFlow
4. Flujo de merges
5. Sistema de Ventas (ventas.sh)
6. Script de automatización semanal (weekly_push.sh)
7. Configuración de Cron
8. GitHub Issues Templates

1. Introduccion

Se nos asignó automatizar el upload de código con un script bash que confirme cambios locales y los envíe a GitHub semanalmente.

También implementamos un sistema de ventas en bash con registro de usuarios, alta de productos y compras, todo sobre GitFlow con ramas feature.

2. Organización del Trabajo

Usamos Jira (kanban) para gestionar las tareas. Dividimos el obligatorio en 19 tareas agrupadas en 3 áreas: sistema de ventas, automatización git, y documentación/issues templates.

Las ramas feature se asignaron por funcionalidad: cada tarea se desarrolló en su propia rama y se integró a develop mediante merges.

3. Inicialización del repositorio y GitFlow

Repo creado en GitHub, inicializado con git init. Ramas:

Rama	Propósito
main	Producción/Versión estable
develop	Integración de features

feature/autenticacion	Registro, login y logout
feature/alta-productos	Alta de productos
feature/venta-productos	Compra de productos

4. Flujo de merges

```
main
├─ develop
│   ├── feature/autenticacion
│   ├── feature/alta-productos
│   └─ feature/venta-productos
```

5. Sistema de Ventas (ventas.sh)

5.1 Menú interactivo

while true, case y read. Muestra estado de sesión.

5.2 Registro de usuarios

registrar_usuario(): pide nombre y contraseña, valida vacíos y espacios, verifica duplicados con grep -q, guarda como USUARIO:nombre clave.

5.3 Inicio y cierre de sesión

iniciar_sesion(): valida credenciales contra el archivo, mantiene variable usuario_logueado.

cerrar_sesion(): limpia la variable.

5.4 Alta de productos

alta_producto(): requiere sesión activa, pide nombre, descripción, precio y stock. Valida campos vacíos y valores enteros.

5.5 Compra de productos

`comprar_producto()`: busca producto con `grep`, verifica stock, actualiza con `grep -v` + archivo temporal + mv.

6. Script de automatización semanal (weekly_push.sh)

En scripts/weekly_push.sh. Automatiza la sincronización del repo local con GitHub.

6.1 Cómo funciona

1. git pull
2. git status --porcelain: detecta cambios
3. Sin cambios: alerta en README.md y termina
4. Con cambios: commitea (sin README.md), hace push
5. Push exitoso: registra éxito + líneas modificadas en README.md
6. Push fallido: registra error en README.md

6.2 Detalle técnico

Problema: si README.md se modifica antes del push, el registro de "éxito" queda aunque el push falle. La solución: git add -A && git reset README.md separa el README del commit. El registro se escribe solo después de saber si el push funcionó.

7. Configuración de Cron

Se configuró una tarea cron para ejecutar el script weekly_push.sh de forma automática todos los domingos a las 09:00 horas:

```
0 9 * * 0 /ruta/completa/obligatorio_taller/scripts/weekly_push.sh
```

La sintaxis de cron sigue el orden: minuto, hora, día del mes, mes, día de la semana.

8. GitHub Issues Templates

Se crearon 4 templates en .github/ISSUE_TEMPLATE/ y se abrieron 8 issues de ejemplo (2 por categoría) en el repositorio:

Archivo	Tipo	Campos
bug.yml	Reporte de error	

		Descripción, pasos para reproducir, comportamiento esperado
feature_request.yml	Solicitud de funcionalidad	Problema relacionado, solución propuesta, alternativas
mejoras.yml	Propuesta de mejora	Área de mejora, beneficio esperado, ejemplo de uso
documentacion.yml	Documentación	Sección afectada, qué falta aclarar, propuesta de contenido

Issues de ejemplo abiertos en: github.com/bocho8/obligatorio_taller/issues